AHU Fault Detection and Diagnosis

Data Scientist (Machine Learning) take-home, VentureSEA

Nauval Zulfikar, November 2026

How to read this report.

- Q1 covers data quality and exploratory analysis. Q2 is a one-page summary of the model; the full work is in `code/notebooks/ahu_fault_detection.ipynb`. Q3 is the production design and Q4 the client reply.
- **Headline result.** Out of fold on January to June, the model finds 27 of 28 labelled faults and gives the right type for 20 to 22 (20 when the alarm thresholds are tuned inside the folds). Median detection delay is 9 to 11 hours. It raises 2 false alarms in 240 AHU-months, and one of the two is a real fault with no work order. The engineering rules a building would use instead find 19 and type 10, with 9 false alarms.
- **Main recommendation.** New units and buildings need a 14-day listening period before their alerts go live.

Question 1: Exploratory data analysis and data quality

1.1 Approach

I worked in five steps. Every check became a function in `code/src/ahu_fdd/clean.py`, so the same rule runs on the test period and later in production. Each step is a section of `notebooks/ahu_fault_detection.ipynb` (sections 1 to 6).

1. **Structure and coverage.** Rows per AHU per day, duplicates, gaps, missing values per column and per building.
2. **Data quality.** Plot every signal per AHU over time, then compare it with its physical range, with its neighbours and with the outdoor air temperature (OAT). All three buildings share the same weather, so OAT works as a reference clock.
3. **Patterns.** Compare operating schedules, valve and setpoint behaviour across buildings and units.
4. **Physics vs labels.** For each fault type, write down which signals should move and why, then measure what moved in the labelled events.
5. **Unlabelled cases.** Look for periods that look like faults but have no work order, and for labelled events that are hard to see.

1.2 Data quality: what I found and what I did

Problem	Where	How I found it	Fix
Logger clock 8 h off for a week	B1, 1 Jun 16:00 to 8 Jun 16:00 (32,153 rows)	Two different readings per timestamp on 1 Jun, then an 8-hour hole on 8 Jun. B1's OAT was out of step with B2/B3: mean gap 3.6 °C as logged, 0.3 °C after shifting by +8 h. The logger was writing UTC.	Shift those rows +8 h
Duplicate export	B3, 14 Feb (3,439 rows)	Identical rows twice	Drop one copy
Fan power in W, not kW	B3-AHU-04, 1 to 21 Apr (6,023 rows)	Power up to 1,000× rated motor power, including 40 "kW" standby readings with the fan off	Divide whole days by 1,000
CO2 stuck at 0 or 5,000 ppm	B2-AHU-03, 20 Jan to 27 Feb (781 rows)	Values at the sensor's range limits	Set to missing
Frozen supply-air sensor (SAT) at 13.1 °C	B2-AHU-08, 21 to 29 Jul, test period (2,587 rows)	Unchanged reading for days while the valve moves	Set to missing and flag. The model gives no alert while a key sensor is flagged.

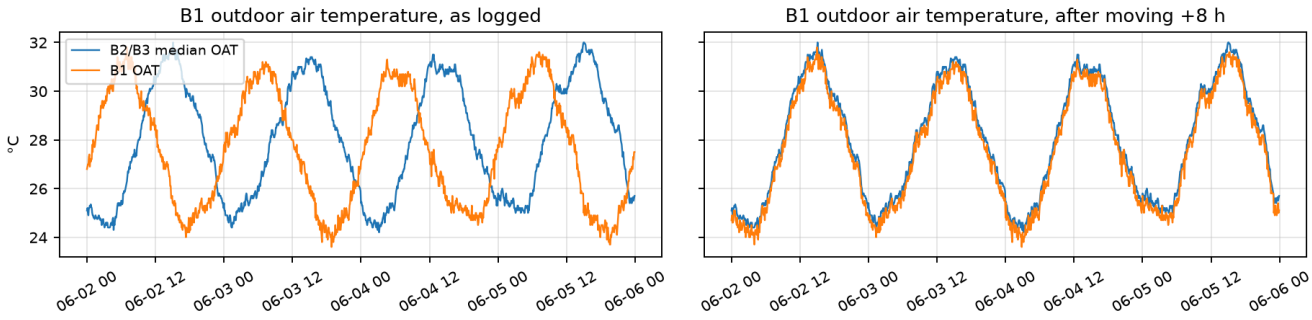
Problem	Where	How I found it	Fix
Frozen return-air sensor (RAT) at 23.4 °C	B1-AHU-07, 5 to 26 May (2,239 rows)	Same check	Same fix
8-day outage	B2, 10 Mar 05:55 to 18 Mar 14:00	Missing rows	Left as a gap. Rolling windows do not reach across it.
Missing by design	CO2 is missing in B3; airflow exists only in B2	Missing values per building	Kept missing. The model treats a missing sensor as missing, not as zero.

Other checks came back clean:

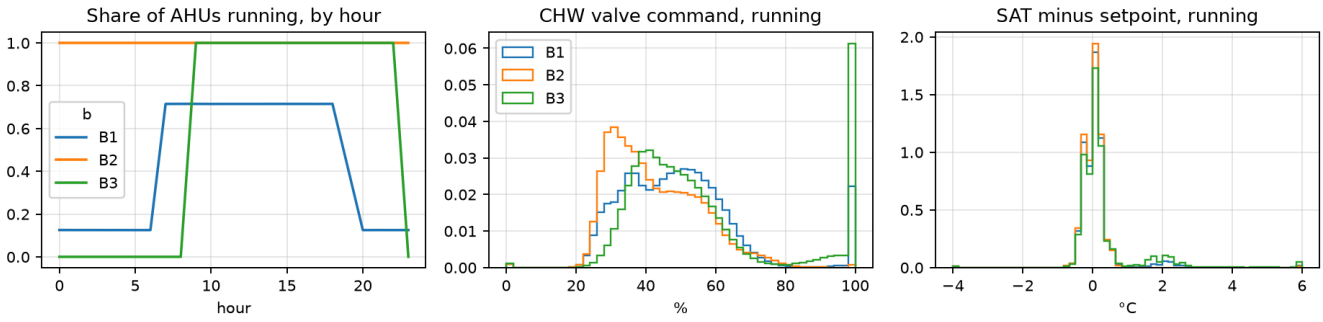
- No other sensor sits at its range limits.
- Setpoints are constant per AHU.
- Fan status agrees with fan speed and power.
- The test period has no missing values. About 0.3% of its expected rows (2,245) are absent: short logging gaps, left as gaps.

What I would ask the client to check:

1. The B1 logger's time zone and NTP setting. If it can drift once, it can drift again.
2. The B3-AHU-04 power meter configuration (units in April).
3. The B2-AHU-03 CO2 sensor, the B2-AHU-08 SAT sensor and the B1-AHU-07 RAT sensor. Frozen sensors are a maintenance issue in their own right.
4. What happened in B2 on 10 to 18 March (BMS or network outage).
5. Three chronically saturated units, B1-AHU-05, B3-AHU-02 and B3-AHU-09 (see 1.3).
6. Filter replacement intervals, B1-AHU-05 in particular.
7. B1-AHU-11, 22 April to 5 May: a probable outdoor-air damper fault with no work order (see 1.5).



1.3 Patterns across buildings and units



Schedules.

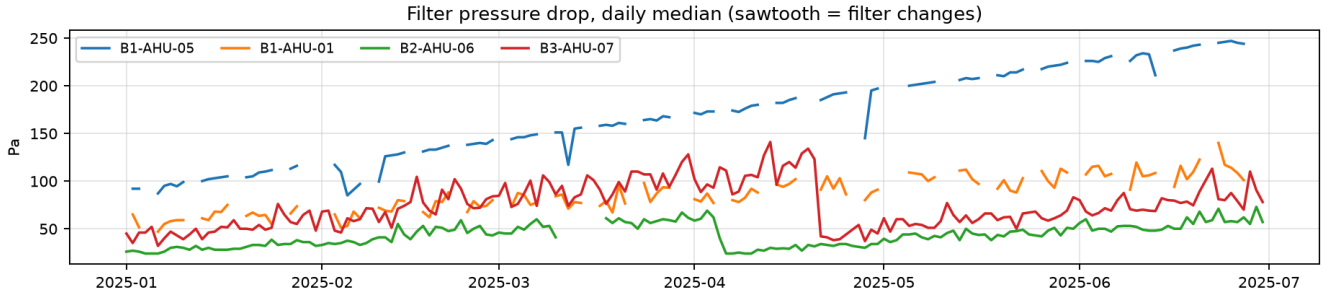
- B2 (hospital) runs non-stop.
- B3 (mall) runs 09:00 to 23:00 every day.
- B1 (office) runs on weekdays only (the curve averages in weekends, hence 72%). Two of its 16 units run 24/7, which gives the 12% floor at night.

Start-up and shut-down rows are transient, so the model scores only steady running rows (fan on for at least 30 minutes).

Valve and setpoint. Most of the time the chilled-water (CHW) valve sits between 25% and 70% and SAT holds setpoint within ±0.3 °C. Two exceptions:

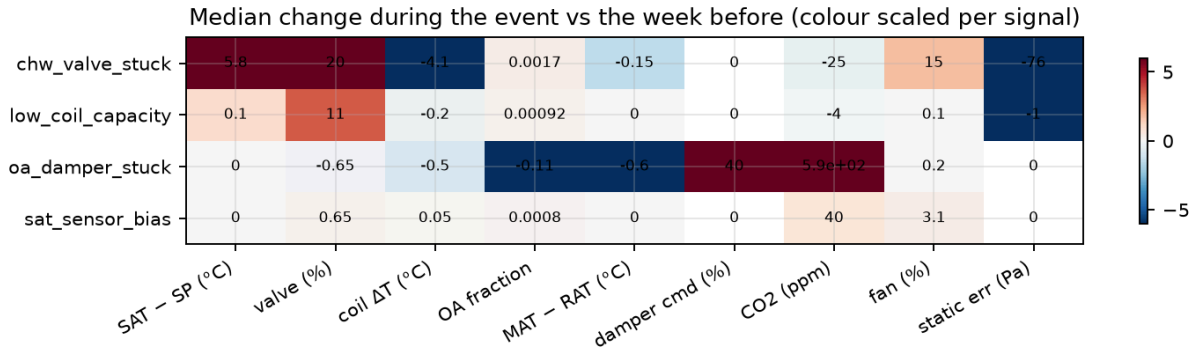
- B3 and B1 show a spike at 100% valve together with a second hump at SAT +2 °C.
- This comes almost entirely from three units: **B1-AHU-05, B3-AHU-02 and B3-AHU-09**. Their valve is above 95% for 73 to 84% of running time and the fan is near 100%, yet SAT still sits 1.5 to 1.9 °C above setpoint, outside any labelled event.
- These units are undersized or chronically degraded (fouled coil, low CHW flow, high filter loss). On them, “low coil capacity” cannot show up as more valve opening. That is why they cause half of the model’s errors (Q2).

Filters. Filter pressure drop follows a sawtooth: a slow rise, then a step down at each filter change (B3-AHU-07 from about 135 to 40 Pa in late April, B2-AHU-06 from about 65 to 25 Pa in early April). B1-AHU-05 climbs from about 90 to 245 Pa over the six months with no change at all, which helps explain its saturation. Filter loading is normal ageing, not a labelled fault, so the normal-behaviour models take filter pressure drop as an input instead of flagging it.



1.4 Fault physics vs the labels

For each labelled event, the figure shows the median of each signal during the event minus the same AHU’s median over the week before. These are steady rows only, with the median taken across events of the same type.



Fault	Expected (physics)	What the labels show
Stuck CHW valve	Stuck closed: less cooling, so SAT rises above setpoint. The controller keeps asking for more, so the valve command rises while coil ΔT falls. The fan speeds up to meet zone load. Stuck open: the reverse (SAT below setpoint, command falls).	SAT error +5.8 °C, valve command +20%, coil ΔT −4.1 °C, fan +15%. Both directions appear, so the classifier needs the sign as well as the size of the change. **Agrees.**
Low coil capacity (fouling, low CHW flow)	Each % of valve opening does less cooling. The valve opens further to hold SAT. SAT stays on setpoint until the valve saturates, then drifts up.	Valve +11%, SAT error about 0. On the three saturated units, SAT rises instead. **Agrees, but subtle:** a slow drift that is easy to confuse with load.
Stuck OA damper (closed)	Less outdoor air, so mixed-air temperature (MAT) moves toward RAT, the outdoor-air fraction falls and CO2 climbs. On CO2-controlled units the damper command rises with no effect.	Outdoor-air fraction −0.11, MAT−RAT −0.6 °C, damper command +40%, CO2 +586 ppm. **Agrees.** In B3, with no CO2 sensor, only the small MAT shift remains.

Fault	Expected (physics)	What the labels show
SAT sensor bias (reads high or low)	The loop controls to the wrong value, so the measured SAT stays on setpoint while the real air is warmer or colder. The only traces are indirect: valve position and fan effort that no longer match the load, and slow comfort effects (CO2, zone load).	Fan +3%, valve +0.65%, CO2 +40 ppm: very small. **Hard to see by design.** It needs an independent temperature to confirm.

1.5 Unlabelled and hard-to-see cases

- **B1-AHU-11, 22 April to 5 May: an unlabelled stuck damper.**
  - The outdoor-air fraction falls from 0.08 to 0.02 and CO2 climbs by about 650 ppm.
  - The damper command rises from 15% to 70% with no effect: the stuck-damper signature exactly, with no work order.
  - The model flags it (Q2), and it is one of only two “false alarms” out of fold. Labels come from work orders, so they are incomplete. A false-alarm count against them overstates the true false alarms.
- **The three saturated units look like permanent low coil capacity.** They are not labelled, and probably should be checked as one maintenance item each.
- **Labelled but hard to see:**
  - Sensor-bias events change almost nothing measurable.
  - Damper faults in B3 have no CO2 to confirm them.
  - Short events (under a day) leave little evidence once the analysis is restricted to steady running rows.
- **Labels have fuzzy edges.** Start and end come from work orders, so signals often begin to move before the labelled start. The model therefore does not train on rows within a day of an event as “normal” (Q2).
- **Test period (July to August):**
  - B2-AHU-08’s frozen SAT is a data fault, not a coil fault. The DQ mask keeps it from raising an alarm.
  - Against their April to June baseline, a few units drift. B1-AHU-12 and B3-AHU-06 run fans about 6% slower, and B2-AHU-12 takes less outdoor air. The model raises sustained alarms on all three in the test predictions (sensor bias, sensor bias and damper respectively). They are worth a site check.

Question 2: Modelling (summary; full work in `code/notebooks/ahu_fault_detection.ipynb`, sections 7 to 21)

Framing. The model works in layers:

1. **Physics features** per row: setpoint error, outdoor-air fraction, coil ΔT, and fan power vs the affinity law.
2. **Fleet normal-behaviour models** (LightGBM, one per signal, AHU as a categorical input), trained on rows with no work order. They predict what each signal *should* be, and the residuals are the fault evidence.
3. **A LightGBM multiclass classifier** (normal + 4 faults) on 3 h and 12 h trailing medians of the residuals and physics features.
4. **A causal alarm state machine.** An alarm opens when P(fault), smoothed over 1 h, rises above 0.8. It closes after 12 h below 0.3. The type is the class with the highest mean probability since the alarm opened.

Why this design? Residuals absorb the unit differences that would force per-unit rule thresholds or teach a raw-signal classifier the unit instead of the fault, and the physics keeps alerts explainable.

Validation.

- **Leave-one-month-out** on January to June with a 2-day purge. Neither the normal models nor the classifier see the scored month.
- **No event leakage.** Events spanning two months leave training in both folds; rows within a day of an event never count as "normal".
- **Scored by event:** found, right type, delay to the first alarm, false alarms per AHU-month.
- **Thresholds** were tuned on the out-of-fold probabilities with `J = typed recall − 4 × FA/AHU–month`.

Results (out of fold, 28 events, 240 AHU-months, 95% intervals). The bold row picks alarm thresholds inside each fold; every other row picks them on the scored months.

	Found	Right type	Median delay	False alarms (per AHU-month)
Model (nested, honest)	27/28 (82-100%)	20/28 (51-87%)	11.3 h	2 (0.001-0.030)
Model (in-sample)	27/28	22/28 (59-92%)	9.3 h	2 (0.008)
Logistic regression	25/28 (72-98%)	17/28 (41-78%)	6.8 h	27 (0.074-0.164)
Expert rules (in-sample)	19/28 (48-84%)	10/28 (19-56%)	2.6 h	9 (0.017-0.071)
BMS alarm (today)	14/28	none	2.9 h	8 (0.033)
No CO2 (or airflow)	28/28 (27)	22/28 (21)	10.6 h	1 (2)
New AHU, day 1	28/28	12/28	0.6 h	179 (0.75)
New AHU, +14 days	28/28	20/28	13.0 h	8 (0.036)

Against the rules, the model finds 8 faults they miss and they find none it misses (McNemar p = 0.008; right type 12 to 0, p = 0.0005), with equal delay on the 19 both find (2.5 h vs 2.6 h). These gaps and the one against the BMS alarm survive a Holm correction. **Against a logistic regression** on the same features the model cuts false alarms from 27 to 2; its typing edge (6 to 1) is not significant (p = 0.125). **Across five seeds** every run finds 27/28, types 21 or 22 and raises 2 or 3 false alarms.

Value in money. Priced with Yuill and Braun's (2017) figure of merit at Indonesian rates (PLN Rp1,115/kWh, technician Rp34,458/h, kW per fault measured), the nested model is worth Rp16,700 per AHU-month against −Rp2,100 for the BMS alarm, the only method above zero in all three scenarios (notebook §18).

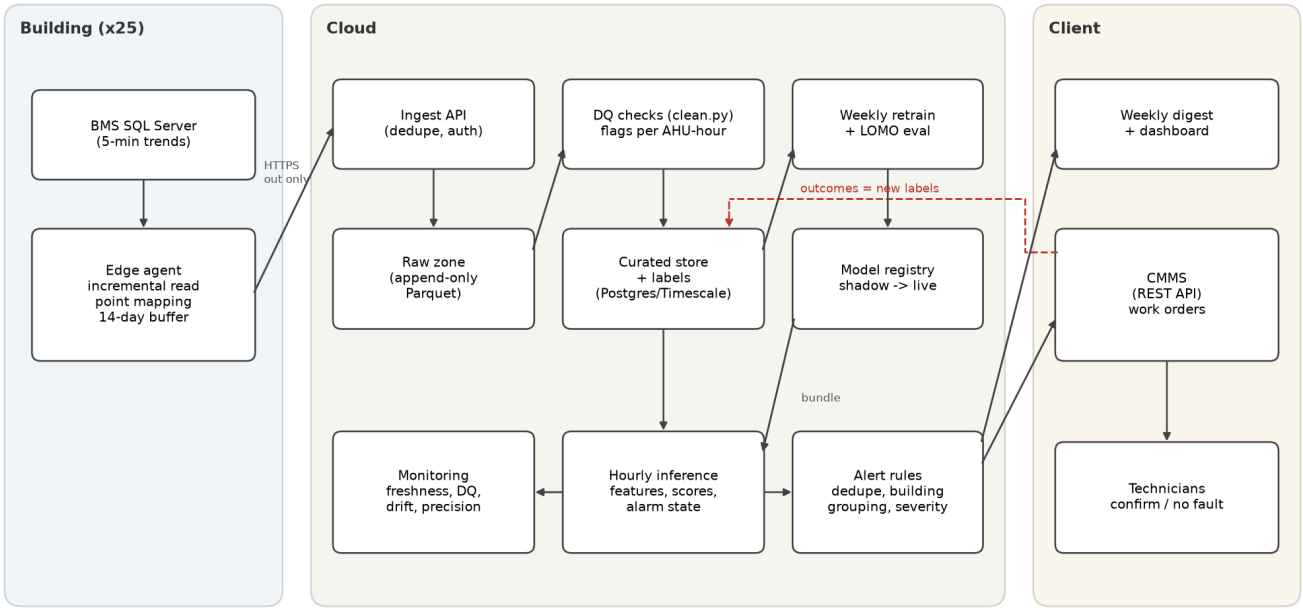
Errors.

- **Half of the 6 misses and wrong types are on the three chronically saturated units** (B1-AHU-05, B3-AHU-02, B3-AHU-09), where valve at 100% makes three faults look alike (SAT just rises). Fix: a saturated-unit mode and a site check.
- **The other confusions** (B1-AHU-02, B3-AHU-06) need an independent discharge or zone temperature.
- **The missed damper fault** (B3-AHU-05) is in the mall, which has no CO2 sensor. **One of the two false alarms is a real but unlabelled stuck damper** (B1-AHU-11, Q1 §1.5).

New AHU and fewer sensors. Missing CO2 or airflow costs almost nothing (the temperature balance carries the damper evidence). **A brand-new AHU is the real risk** (179 false alarms on day one); **fourteen days of listening** with the normal models refit fixes most of it, which drives onboarding in Q3 and Q4.

Test period. `predictions.csv` (711,994 rows) passes the validator and holds 12 sustained alarms (over 6 h).

Question 3: Production architecture



3.1 Data path

Edge (one small agent per building). A Windows service reads new rows from the BMS SQL Server every 5 minutes. It uses the timestamp plus the row key as a watermark, so each read is incremental. It maps the site's point names to the standard schema I define using a per-building mapping file (a YAML file kept in Git), writes compressed Parquet into a local buffer, and pushes the files over outbound HTTPS to an ingest API. The agent does no modelling. Because the PC only allows outbound traffic, the agent also polls the API for config and version updates, so I never need inbound access. The buffer holds 14 days, so an internet outage loses nothing: the agent re-sends when it reconnects and the ingest API ignores duplicates by (ahu_id, timestamp, hash). The agent sends a heartbeat every 5 minutes.

Cloud storage. Files land in object storage in two zones:

- *raw*: append-only and never edited, so any result can be replayed.
- *curated*: the output of the same `clean.py` used in this take-home, run as data-quality (DQ) checks. These cover clock shifts, duplicate exports, unit errors, out-of-range values, frozen sensors and gaps. Each check writes a DQ flag per AHU per hour instead of silently fixing data. Flags feed both the model, which gives no fault alert while a key sensor is flagged, and a DQ ticket queue, because a frozen sensor is itself worth a work order.

Scale. 600 AHUs at 12 rows an hour is about 173k rows a day, or about 2,000 AHUs and 576k rows a day at the stated target. That is a few hundred MB a day in Parquet. It fits comfortably in one Postgres/TimescaleDB instance or DuckDB over Parquet. I do not need Spark or streaming: an hourly batch job on one machine handles the whole fleet in minutes.

3.2 Models

Training (weekly, or on demand). The job retrains the fleet normal-behaviour models and the fault classifier on all curated history, with no-fault rows defined as rows without an open work order. It runs the same leave-one-month-out evaluation as the notebook. The output is one versioned bundle: model files, alarm thresholds, feature list, code commit and evaluation report, stored in a model registry (MLflow or a versioned bucket).

Promotion. False alarms are rare (0.008 per AHU-month in Q2), so a one-week shadow run on 40 AHUs covers about 9 AHU-months and expects fewer than one false alarm from either bundle. A week cannot tell two bundles apart, so promotion has two gates. First, a *labelled replay*: the new bundle (challenger) and the live one (champion) score all labelled history with the same out-of-fold procedure, and the challenger must find and type at least as many events, compared event by event with McNemar's test, with no more false alarms. Second, a fleet-wide *shadow run* long enough to bound the false-alarm rate. With no unconfirmed alarm in 30 AHU-months, the 95% upper bound on the rate is 0.1 per AHU-month, which takes a little over three weeks on 40 AHUs and under two days on 600. Rollback means pointing back to the previous bundle ID, which takes minutes because bundles are never edited. The Q2 notebook writes such a bundle (`outputs/model/bundle.json` holds data hashes, library versions, seed, thresholds and code commit).

Inference (hourly). For each AHU the job takes the last 24 h of curated data, computes features, scores, and runs the alarm state machine. Its state (open alarms, cumulative type probabilities) persists between runs, so the logic is exactly the causal code validated offline. Hourly is enough because these faults develop over hours and cost money over days: the validated median delay is about 10 hours, set by the 3-12 h evidence windows, not by how often I score. Scoring every 5 minutes would add cost and alert churn and find nothing sooner. Safety-critical conditions (a fan trip, a fire damper) stay with the BMS's own alarms.

Technology choices. Python, LightGBM and Parquet, because that is what the model is built with and one engineer can run it. Postgres/TimescaleDB for curated data and alarm state, because the volume is small and SQL suits the facility engineers. A scheduler (Prefect, or cron on a managed container) runs the jobs. A managed cloud (Azure fits sites that already run Windows/SQL Server) means no servers to patch. Nothing in this design needs Kafka, Spark or a GPU.

3.3 From score to work order

Raw alarms do not go straight to the CMMS. They pass through a small rules layer:

1. **Deduplicate.** One open ticket per AHU and fault. A repeat alarm updates the existing ticket instead of opening a new one.
2. **Group by building.** If more than about 30% of a building's AHUs show the same signature at once (for example SAT high with valves at 100%), it is probably a plant problem such as warm chilled water. The layer raises one plant ticket and suppresses the AHU tickets.
3. **Severity and route.** Faults that waste energy or affect comfort now (stuck valve, stuck damper on a CO2-controlled unit) become work orders. Slow faults (coil capacity, sensor bias) go into a weekly digest. Each ticket carries the fault type, the evidence plots, and a short "what to check" note per fault type (e.g. *stroke the valve 0 to 100% and watch SAT*).
4. **CMMS REST API.** Tickets are created and updated through the client's CMMS. When a technician closes a ticket, the result (*confirmed* / *no fault found* / *other cause*) is read back. **These outcomes become new labels** and drive the precision metric. Without them the system cannot get better.

3.4 Monitoring

What	How	Alert when
Data freshness	heartbeat and last row per building	no data for 30 min
Data quality	DQ flags per AHU per day	new flag type, or > 5% of rows flagged
Input drift	PSI of the 10 most important features and of weather and chilled-water temperature, live month vs training window	PSI above 0.2 (watch) or 0.3 (act), read against the largest month-to-month PSI seen in training
Alert volume	alerts per AHU per month, per building	2x the validation rate
Outcome	precision from CMMS outcomes, time to close	precision below 60% over 4 weeks

Retraining triggers. Besides the weekly job, a retrain starts early when a key feature crosses PSI 0.3 in more than one building, when precision from CMMS outcomes falls more than 5 points below its level over the previous quarter for 4 weeks, when 10 or more new confirmed labels have arrived, or when a building changes its schedules or setpoints. A retrained bundle still has to pass both promotion gates.

First month after go-live: every alert is reviewed by an engineer before it reaches the CMMS, every no-fault-found outcome is reviewed with the technician within 48 h, and these numbers go to the client weekly. A rising alert rate with no change in confirmed faults usually means a building has changed (new schedule, setpoints, BMS update), not that the units broke. The response is to re-baseline that building, not to retune the model for everyone.

3.5 Onboarding a new building (connected in under a week, alerts after two weeks)

1. **Day 1:** install the agent and fill the point-mapping template. The agent validates the mapping against the BMS point list and reports any missing points.
2. **Days 1-2:** backfill as much history as the BMS keeps (usually 3 to 12 months). Run the DQ checks on the backfill and review them with the site's engineer. This is where wrong clocks and wrong units turn up.
3. **Days 2-5:** score the building with the fleet models in shadow mode. A new AHU starts from the fleet models, which already know buildings of the same type. Without its own history, though, it raises about 0.75 false alarms per AHU-month in Q2's test, roughly 90× the rate on known units (0.008). So the normal models are refit once the unit has 14 healthy days. In Q2 that brings the rate down to 0.036 per AHU-month.

4. **Go-live** after two weeks in shadow mode with at most one unconfirmed alert per building. On 12 to 16 units, two weeks is only 6 to 8 AHU-months, so this check catches a badly mapped building but cannot prove a rate of 0.1 false alarms per AHU-month. The rate is then tracked monthly on the whole fleet. Until go-live, an engineer reviews every alert before it reaches the CMMS.

Missing sensors are handled by design. The feature set already distinguishes buildings with and without CO2 and airflow, and Q2 shows that detection barely changes without them (28 found and 22 typed with CO2 removed, against 27 and 22). A building with fewer points gets the same model with those inputs left empty, plus a lower confidence label on damper tickets, since the one missed damper fault was in the building without CO2.

Question 4: Client communication

4.1 Reply to the facilities manager

Subject: Last week's 9 alerts, and what happens before the next buildings

Dear [Name],

Thank you for telling me straight. You are right to be frustrated. Four call-outs that found nothing is four wasted trips, and if your team stops trusting the alerts, the system is not doing its job. That is my problem to fix, not your technicians'.

Here is what I will do, starting this week:

1. **I go through all 9 alerts with you.** For each one I will show what the system saw and tell you plainly whether it was my mistake. Some faults, like a temperature sensor that reads 2 °C high, cannot be seen by walking up to the unit. For those I will give your technicians a 10-minute check to confirm or rule it out. The rest I will treat as false alarms and fix.
2. **For the next two weeks, I check every alert before it reaches your team.** Your technicians will only get alerts I am confident about, each with a short note saying what to check and where. I expect that to be one or two a week for this building, not nine.
3. **You get a one-page summary every Friday:** alerts sent, what your team found, and what I changed.

For the building owner, I suggest this message: *the pilot is working but still being tuned; the two new buildings will start next month in "listening mode", where I collect their data and check it myself, with no alerts to technicians; alerts go live there once this building has had two clean weeks.* The owner keeps the timeline, and your team gets no extra load until the system has earned it.

Could I have 45 minutes with you and one of the technicians who did the checks, on Tuesday or Wednesday? Their notes on what they found will help me more than anything else.

Kind regards,
Nauval Zulfikar
Data Scientist, VentureSEA

4.2 Internal note to the project lead

What I think is happening. In validation the model raised about one alarm per AHU every seven months, almost all of them real faults. For a building of 12-16 AHUs that is roughly one alert every two weeks, so 9 in one week is about 20 times what I expected. Something is different in this building, not just unlucky. The likely causes, from most to least likely:

- **The building's baseline is probably too short.** If the normal-behaviour models only know this building from its first two weeks, that is too little; in validation every unit had six months, and Q2 shows a unit with no history raises about 90 times more false alarms than a known unit. A short baseline sees normal swings (a rainy week, a schedule change) as faults.
- **One root cause raised several alerts.** If the central plant's chilled water ran warm, every AHU looks like it has a weak coil. That is one plant ticket, not five AHU tickets.
- **Data problems were read as faults.** In the training data alone I found a logger on the wrong clock, a frozen sensor and a power meter in the wrong units. Any of these in the pilot building would trigger alerts.
- **Some "nothing found" may be real faults** that a visual check cannot see (sensor bias, a valve stuck at 30%). I would not assume all 4 were false.

Next two weeks.

- Days 1-3: triage all 9 alerts against the raw trends; label each one *real fault* / *data issue* / *plant-level* / *false alarm*. Check the four "no fault found" units with a reference thermometer and a valve stroke test.
- Days 3-5: add a building-level grouping rule (same signal moving on most AHUs at once means a plant ticket), add DQ gating (no fault alert while a sensor is flagged), and refit the thresholds for this building on everything collected so far.
- Weeks 1-2: human review of every alert before it reaches the CMMS. Track precision and the false alarm rate per AHU-month daily.

What I need from the client. Work-order notes for all 9 alerts (what was checked, how); 45 minutes with a technician; the BMS point list and any changes made since go-live; notice of planned maintenance and chiller plant events.

Rollout recommendation. Do not go live with alerts in the two new buildings next month. Connect them now and run them in listening mode (data, DQ checks and baselines, no alerts). Turn alerts on when the pilot meets two weeks with at least 70% of alerts confirmed and fewer than 0.1 false alarms per AHU per month. On current evidence that is four to

six weeks later than planned, and it means the new buildings start with a month of baseline history instead of two weeks, which fixes the main cause of this problem before it happens again.